

Dobot Magician E6 ultrasound teleoperation

How to connect, run, drive and pack down the arm for live probe-on-phantom demonstrations. Written for engineers who have not worked with a robot arm before.

The arm is position controlled and does not feel you. It will not stop for your hand, the bench, or the phantom. Keep the emergency stop within reach and read section 1 first.

Robot Dobot Magician E6
Controller PS5 DualSense
Stack ROS 2 Humble + MoveIt Servo
Arm address 192.168.5.1
Revision 2026-10-04

Contents

- 01 Safety and the emergency stop
- 02 Status lights
- 03 Unpacking, packing and connecting
- 04 Running and stopping the program
- 05 Joystick controls
- 06 The probe: payload and contact force
- 07 Limits, faults and recovery
- 08 Running on Windows
- 09 Reference

Safety and the emergency stop

Read this before touching anything. The rest of the manual assumes you know where the stop button is.

The arm does not feel you

⚠ This robot is **position controlled**. It moves to wherever the joystick tells it to go and keeps pushing to stay there. It does **not** stop when it meets your hand, the bench, or the phantom. There is no force sensor fitted.

Treat every commanded move as something that will happen regardless of what is in the way.

The emergency stop

The red mushroom button on a cable from the robot base is the only control that stops the arm unconditionally. Press it and the arm halts all motion and locks.

- It **latches down** when pressed — it stays in until you release it.
- To release, **rotate** the button in the direction of the arrow marked on it.
- Alarms cannot be cleared and the arm cannot be enabled until it is released.
- Keep it within arm's reach of the operator, not buried behind the laptop.

The deadman principle

Nothing moves unless **L1** is held down. This is deliberate: dropping the controller, letting go in a panic, or walking away all stop the arm.

Releasing L1 should be your first reflex — it is faster and gentler than the mushroom button. Save the emergency stop for when the arm is doing something you did not command.

Rules for the stand

Visitors **may** drive the arm — that is the point of the demonstration — but only under direct supervision, and only within limits.

Before handing the controller over

1. Show them **L1**: "hold this to move, let go and it stops."
2. Tell them to let go the moment anything surprises them.
3. Keep the controller tethered to your stand, or keep a hand on the cable.

While a visitor is driving

- **Stay beside them with a hand near the emergency stop.** Supervision means watching the arm, not chatting over your shoulder.
- **Visitors drive in free space only.** *You* make any approach to the phantom — the arm cannot feel contact and will keep pressing. See [§6 Contact force](#).

- Nobody puts a hand inside the arm's reach while the program is running. It sweeps a **450 mm radius** sphere around its base.
- Keep the phantom, gel and probe cable clear of the arm's path.
- If anyone is unsure what the arm is about to do, **release L1**. That is always safe and always stops it.

Turbo is an operator control

Do not mention or demonstrate R1 to visitors. Turbo multiplies every rate by **1.8**, and the whole reason visitor driving is safe is that the base rate is slow enough to watch and react to. Visitors should finish the demonstration without knowing the button exists.

Keep it for yourself when you need to reposition the arm quickly between visitors — and never for an approach toward the phantom or a person.

If you would rather remove the risk entirely, set `scale_turbo` to `1.0` in `joy_params.yaml` and restart. See §9.

Things that are safe, and often mistaken for faults

A clicking sound when the arm enables	Joint brakes releasing. Normal and unavoidable
A fan running constantly	The control computer lives in the base
The arm stopping dead mid-move	Usually a joint limit or singularity. See §7
The controller buzzing	Singularity proximity warning, not contact force

Status lights

A ring of LEDs sits near the base joint and near the wrist. It is the fastest way to know what the arm is doing.

LIGHT	MEANING	WHAT TO DO
● Blue, fast flash	Booting	Wait 30–60 s
● Blue, steady	Powered, not enabled	Normal with the program stopped. Ready to launch
● Green, steady	Enabled and idle	Normal just after launch
● Green, slow flash	Running under remote commands	Normal while teleop is live
● Green, fast flash	Jogging, drag mode, or recording	Not used by this setup — investigate
● Yellow	Joint brakes engaged while disabled	Usually harmless; enable or restart
● Red, steady	Alarm, or collision detected	Stop. Troubleshooting →

Fast flash is about 5 per second, slow about 1 per second.

The two you will see all day

Steady blue	Program stopped, arm resting
Slow-flashing green	Program running, arm under your control

Anything **red** means an alarm — stop and read [Troubleshooting](#).

The blue and green meanings are confirmed on this arm. The exact yellow and red pairings were read from a table in the manufacturer's PDF and may differ slightly on your unit — but any red light means an alarm, which is all you need in order to act.

Unpacking, packing and connecting

The arm travels folded into a **packing posture**, and getting into and out of that posture needs **DobotStudio Pro on the Windows laptop**. Do that first — the Linux laptop is not involved until the arm is already unfolded.

Three different poses — do not confuse them

POSE	WHAT IT IS	HOW YOU REACH IT
Packing posture	A compact fold that fits the foam in the box	DobotStudio Pro → RunTo
Home pose	The upright starting posture	DobotStudio Pro → Home pose
Our teleop home	<code>[0, -19.24, 117.25, -8.02, -90, 0]°</code> , flange parallel to the bench	Automatic — the controller combo, or <code>go_home.py</code>

The first two are Dobot's own, reached only through their software. The third is this project's, and is what the arm uses during a demonstration.

The DobotStudio Pro procedure

The same procedure both unfolds and folds the arm — only the final button differs. It runs on the **Windows laptop**, not the Linux one.

1. Connect the **Windows laptop** to the robot by Ethernet.
2. Open **DobotStudio Pro**.
3. Connect to the robot using **EtherCAT**.
4. **Enable** the robot.
5. Go to **Settings → Posture**.
6. **Set the speed below 40%**. Do this before moving anything — the arm folds through a large range and you want time to react.
7. Then, depending on what you are doing:

	BUTTON	ACTION
Unpacking	Home pose	Click and hold until the arm arrives
Packing	RunTo	Click and hold until the arm arrives

1. **Disable** the robot.
2. Disconnect the Windows laptop and move the Ethernet cable to the **Linux laptop**.

The click-and-hold is a deadman. The arm moves only while the button is held, and stops the moment you let go. If anything looks wrong, release.

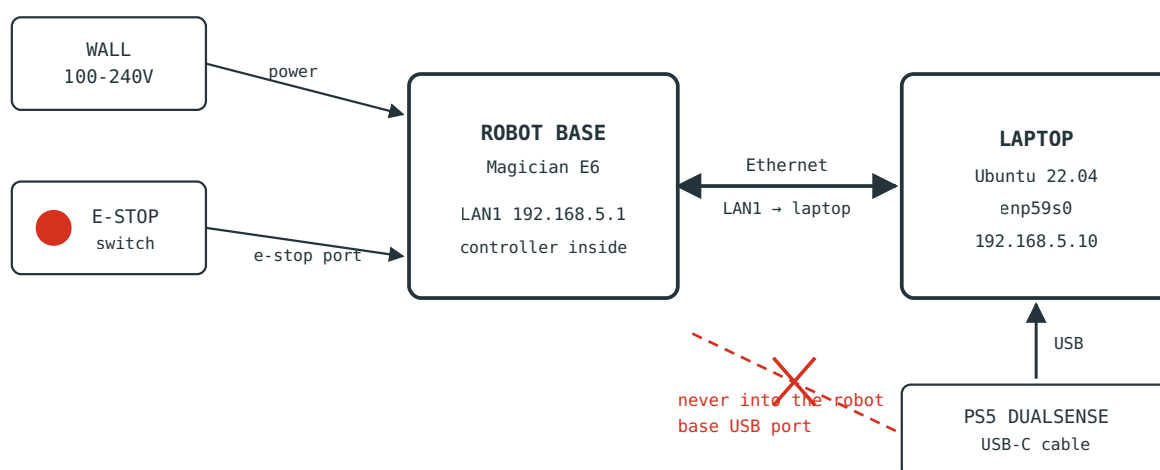
⚠ Keep a hand near the **emergency stop** throughout. The arm is bolted to the bench and the fold sweeps a wide path — check it will not drive into the benchtop, the phantom, or a cable.

Unpacking

1. Open the box flat on the floor or a sturdy table. The arm weighs **7.2 kg** and is awkward — **use two people**.
2. Lift by the **base and upper arm together**, keeping the folded posture it was packed in. Do not lift it by the forearm or the wrist.
3. **Bolt the base down before doing anything else**. An unbolted arm will tip itself over the moment it moves.
4. Keep the box and foam — you need them to get home again.
5. Connect the cables (below) and power on.
6. Run the **DobotStudio Pro procedure** above, choosing **Home pose**.

⚠ The arm reaches 450 mm and can move its tool at 0.5 m/s. A 7.2 kg arm doing that on an unsecured base will walk off the bench. **Mount it before powering on.**

The connections



ORDER	CABLE	FROM	TO
1	Emergency stop	E-stop switch	Robot base e-stop port (usually pre-fitted)
2	Power	Robot base	Wall socket — leave switched off for now
3	Ethernet	Robot LAN1	Laptop wired port
4	USB-C	PS5 controller	Laptop

The controller plugs into the laptop, never into the robot. The USB socket on the robot base is for a WiFi dongle or file transfer only — a controller plugged in there does nothing.

LAN1 is the port fixed at 192.168.5.1. LAN2 is a different subnet and will not work. Connect everything **before** switching on at the wall.

Network configuration for the Linux laptop is in [§4 Running and stopping](#).

Powering the arm on

⚠ **Tap it — do not hold it.** A short press **under 0.5 s** switches the arm **on**. **Holding for 1.5 s is the shut-down gesture.** If you press and hold you will see the light flash and then die when you let go — that is the robot booting and then being told to switch off. Every other button on the robot uses long-press to activate; the power button is the exception.

1. Check the e-stop is **not latched**. If pressed in, rotate to release.
2. Switch on at the wall.
3. **Quick tap** the power button on the base. Finger off immediately.
4. Watch the LED ring: blue fast flash (booting, 30–60 s) → **steady blue**.

Steady blue is the ready state — powered and healthy, waiting for the program. See [§2 Status lights](#).

Packing down at the end

1. **Stop the program** with `Ctrl+C` on the Linux laptop. DobotStudio and this software cannot both control the arm.
2. Remove the probe and any tooling from the flange.
3. Move the Ethernet cable to the **Windows laptop**.
4. Run the **DobotStudio Pro procedure** above, choosing **RunTo**.
5. **Disable** the robot in DobotStudio.
6. Power off: **hold** the power button for 1.5 s until the light flashes red fast, then release.
7. Switch off at the wall and disconnect the cables.
8. Unbolt the base.
9. Lift with two people, keeping the folded posture, and lower it into the foam.

⚠ **Do not transport it in any other pose.** The manufacturer requires the packing posture with working brakes before it goes in the box — the brakes are what hold the pose once power is off. Transporting it folded wrongly risks damaging the joints, and the foam will not support it.

Running and stopping the program

Everything runs from one terminal on the **Linux laptop**. There is **no timer and no auto-shutdown** — it runs until you stop it.

1. Network

Needed after every laptop reboot. The wired adapter carries a static address on the robot's subnet with **no gateway**, otherwise traffic for the robot leaves over WiFi instead.

```
bash nmcli con up dobot-e6 ping -c3 192.168.5.1 # must reply before going further
```

If the profile is ever missing, recreate it once:

```
bash sudo nmcli con add type ethernet ifname enp59s0 con-name dobot-e6 \ ip4 192.168.5.10/24
ipv4.never-default yes
```

2. Load the environment

```
bash cd "/home/yz22/Documents/code/Dobot arm" source setup_dobot.bash
```

3. Send the arm to its teleop home

Run this **before** launching. It needs the robot's command port, which the running program holds open — so it will not work once the launch is up.

```
bash python3 src/dobot_e6_hw/scripts/go_home.py --speed 5
```

It counts down from three before moving, prints joint angles as it goes, and travels slowly. The click as it starts is the joint brakes releasing — normal whenever a disabled arm is enabled.

4. Launch

```
bash ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1
```

What a good startup looks like



Watch for these four lines:

```
[dobot_tcp_node]: RequestControl → 0, {}, RequestControl(); [dobot_tcp_node]: EnableRobot → 0,
 {}, EnableRobot(); [activate_servo]: Servo started – hold L1 and move the sticks to drive.
[haptic_feedback]: Calibration complete – peak_cond=16.5 LOWER=24.8 HARDSTOP=44.7
```

Wait for the slow-flashing green light

Do not touch the controller until the LED ring is flashing green slowly. That is the arm's own confirmation that it is enabled and accepting remote commands — about 15 seconds after launch.

Then keep your hands off for a further **25 seconds**, until `Calibration complete` appears. That window measures how the arm behaves at rest to tune the rumble warning. Move the arm during it and the warning will be mis-tuned — you will get buzzing at rest, or no warning when you need one.

Allow about **40 seconds** from launch to driving. [More on the lights →](#)

`RequestControl()` is what claims TCP control of the arm. It runs at **every** startup because the grant does not survive a power cycle. No DobotStudio Pro and no Windows machine is needed for this part.

Stopping

Press **Ctrl+C** in the terminal running the launch and wait for the prompt to come back. The software disables the arm on the way out, so the LED returns to **steady blue**. That is the clean, finished state — there is nothing else to shut down.

Homing while the program is running

Use the controller: hold **L1 + L2 + R2** for one second.

`go_home.py` only works with the launch stopped, so it is for before you start or after an error — not during a demonstration.

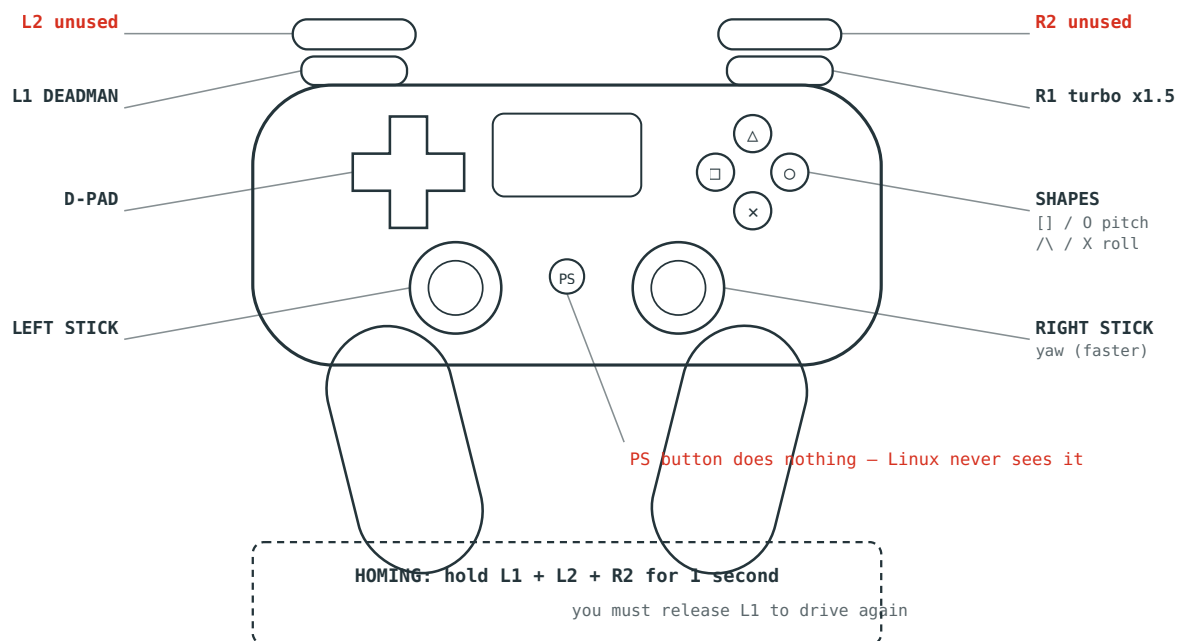
Health checks (optional)

Only needed if something looks wrong. Run in a **second** terminal with the program still going:

```
bash source setup_dobot.bash ros2 topic echo /servo_node/status # 0 = no warnings ros2 topic echo /joint_states --once # do the angles match the real pose? ping -c3 192.168.5.1 # network alive? ls /dev/input/js0 # controller present?
```

Joystick controls

Hold L1 or nothing happens. Every direction is described from the operator's point of view, standing in front of the arm and facing it.

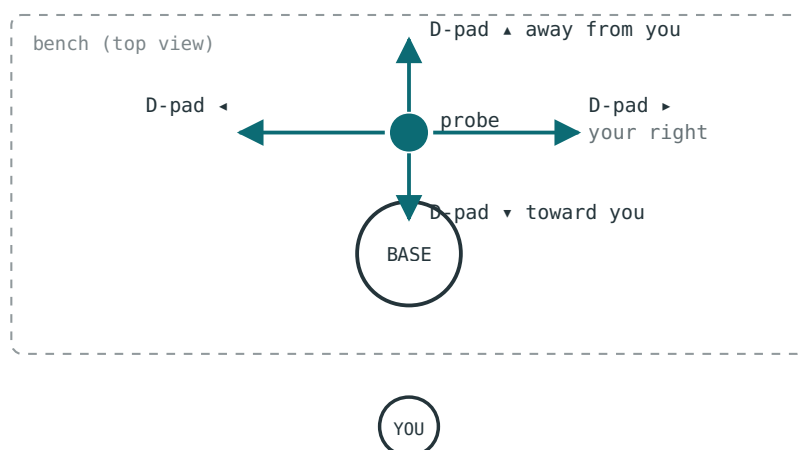


What each control does

CONTROL	MOTION	RATE	FEEL
L1	Deadman — hold it or nothing moves	—	on / off
D-pad ◀ ▶	Move to your left / right	0.03 m/s	full rate or nothing
D-pad ▲ ▼	Move away from / toward you	0.03 m/s	full rate or nothing
Left stick ▲ ▼	Move up / down	0.03 m/s	proportional
Right stick ▲ ▼	Yaw — twist about vertical	0.3 rad/s	proportional
□ / ○	Pitch — tilt the probe nose	0.2 rad/s	full rate or nothing
△ / ×	Roll — rotate about the probe axis	0.2 rad/s	full rate or nothing
R1 (with L1)	Turbo — <i>operator only, see §1</i>	×1.8	hold
L1 + L2 + R2	Home — return to the safe pose	slow	hold 1 s
L2 or R2 alone	Nothing	—	—
PS button	Nothing — not visible to the software	—	—

The PS button is unusable: the index exists in the controller's button array but the press never reaches the software, because something upstream grabs it. It was deliberately left unused.

Directions are from where you stand



Stand **in front of the arm, facing it**. The D-pad then moves the probe the way the arrow points, from your point of view — so your left is the robot's right.

If you walk round to the side of the bench, the mapping no longer matches your body. Move back to the front before driving.

Homing

Hold **L1 + L2 + R2 together for one second**. The arm pauses, moves gently to the safe pose, and stops.

Afterwards, teleop stays disabled until you release L1 and press it again. This is deliberate — it stops a stick you are still holding from flinging the arm the instant homing finishes. If the arm seems dead after homing, this is almost always why.

Homing moves joint by joint, so it works even from a pose that ordinary driving cannot escape. But it takes the **direct route with no collision checking** — clear the phantom out of the way and keep a hand near the e-stop.

The rumble

The controller vibrates as the arm approaches a **kinematic singularity**. Stronger buzz means closer. [See §7 →](#)

⚠ **The rumble is not contact force.** No force sensor is fitted. The vibration describes the arm's *geometry*, not how hard the probe is pressing. You will feel nothing when the probe touches down.

Re-mapping

The mapping lives in `src/dobot_e6_bringup/config/joy_params.yaml`. Edit and restart the program — no rebuild needed.

Every index in that file was **measured** on this specific controller, not inferred from documentation. If you change controllers, measure again rather than assuming: the kernel and the ROS driver disagree about D-pad sign, and an earlier version of this project had a trigger mapped so that the arm climbed at full speed the instant the deadman was pressed.

The probe: payload and contact force

The two questions that matter most for scanning: what changes when you fit the probe, and what happens when it presses on the phantom.

Fitting the probe

Mechanical — stay under 750 g

The E6's rated maximum load is **750 g**, and that budget covers **everything** you bolt on: probe, adapter plate, bolts, and any cable weight the arm actually carries. A typical clinical probe is 200–500 g, so there is room — but weigh the assembled tool rather than guessing.

- Mount to the **four M6 threaded holes** in the end flange.
- Use the 6 mm locating hole with a dowel pin if you need repeatable alignment.
- **Strain-relieve the probe cable** to the forearm. A cable that snags mid-scan drags the arm off course, and the arm will not notice.
- Keep the probe's mass close to the flange. Weight hung far out loads the wrist joints much harder than the same weight held close.

Software — tell the controller about the load

⚠ **Not yet configured — action needed before the exhibition.**

The program currently enables the arm declaring a payload of **0 kg**. That was deliberate during commissioning: a zero payload skips the controller's load check, which otherwise refuses to enable when the configured weight does not match reality.

With a probe fitted and the payload still zero, the controller's internal model of the arm is wrong. The practical consequence is **collision detection firing spuriously** — the arm reads the probe's weight as an unexplained force — plus slightly degraded gravity compensation.

The fix is one `SetPayload()` call with the real mass and its offset from the flange, wired into startup. **This has not been implemented yet.** Weigh the probe assembly first.

Re-home after fitting

Stop and restart the program once the probe is on. The haptic calibration measures the arm at rest, and it should do that with the final tool attached.

Contact force on the phantom

⚠ **The arm does not yield.** This is the most important thing in this manual.

The arm is **position controlled**. When the probe meets the phantom it does **not** sense the contact and does **not** back off. It keeps driving toward the position your joystick commanded, pressing harder the further you push past the surface. It behaves like a clamp, not like a hand.

What actually happens when you press down

1. You hold **L1** and push the left stick down. The arm descends at 0.03 m/s.
2. The probe touches the phantom. **Nothing changes** — no resistance is felt by the software, no rumble, no slowing.
3. Keep pushing and the arm keeps descending, deforming the phantom and loading the probe.
4. Eventually either the controller's **collision detection** trips and raises an alarm, or it does not — and the force keeps climbing.

Step 4 is the uncomfortable one. Collision detection is a safety backstop with a threshold, designed to notice a crash, **not** to regulate scanning pressure.

How much force can it generate?

Dobot does not publish a maximum force figure for the E6. The 750 g load rating is *not* a force limit — it describes what the arm can carry and accelerate, and a position-controlled arm pressing into a rigid object can generate substantially more than the weight it is rated to lift.

Treat the pressing force as **unknown and potentially much higher than you expect**.

How to scan safely, given all of the above

- **Approach in small taps.** Tap the stick down, release, look. Do not hold it down and watch.
- **Release L1 the instant the probe touches.** That freezes the arm.
- **Judge pressure by eye** — watch how far the phantom surface deforms. That is your only feedback.
- **Never put a hand between probe and phantom** to check contact.
- **Do not use turbo** for approach moves.
- Use a **compliant phantom** if you have the choice. Against something rigid, a small over-travel becomes a large force very quickly.

If you need real force control

Proper scanning pressure needs a **wrist force/torque sensor**. The software already has a contact-force rumble cue written and waiting — it needs a sensor publishing on `/ft_sensor/wrench`, and none is fitted. With that hardware the arm could hold a target *force* instead of a target *position*.

That is the correct engineering answer for scanning, and it is a hardware addition, not a software setting.

Limits, faults and recovery

What to do when the arm stops, misbehaves, or shows a red light. Start at the top — most "faults" are the arm behaving correctly at a limit.

First: a stopped arm is usually not a fault

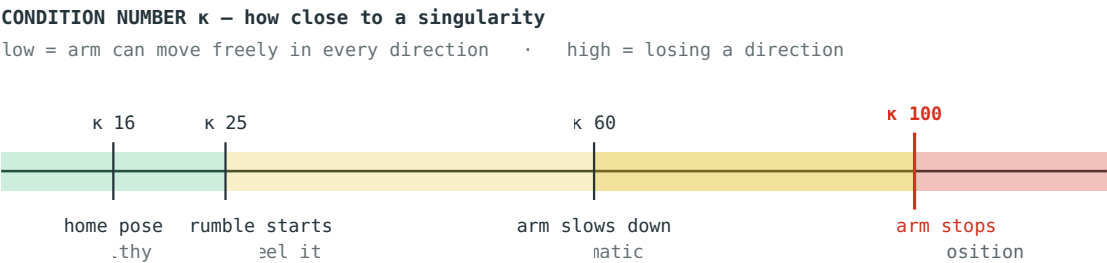
- ✓ When the arm reaches a joint limit or a singularity it **stops moving and holds position**. It does not shut down, power off, drop, or raise an error. Motion simply stops being commanded. **No restart is needed** — drive back out the way you came.

Check this before assuming something is broken. The giveaway is a **steady or slow-flashing green** light and no error in the terminal.

Singularities

A singularity is a pose where the arm loses the ability to move in some direction, typically when joints line up. Near one, small tool movements demand enormous joint speeds, so the software intervenes early.

Closeness is measured by the **condition number (κ)**: low is healthy, high is dangerous.



K	WHAT HAPPENS
~16	The home pose. Healthy
~25	Controller rumbles. Re-measured at every launch, so the exact figure varies
60	The arm automatically slows down
100	The arm stops and holds position

You get three escalating warnings before anything stops.

Joint limits

The software stops commanding motion **0.1 radians (about 5.7°) before** a joint reaches its limit, so the arm never runs into its own hard stops. You will also feel rumble as a joint nears its limit.

JOINT	TRAVEL	JOINT	TRAVEL
J1 base	$\pm 360^\circ$	J4 wrist 1	$\pm 160^\circ$
J2 shoulder	$\pm 135^\circ$	J5 wrist 2	$\pm 173^\circ$
J3 elbow	$\pm 154^\circ$	J6 flange	$\pm 360^\circ$

Getting unstuck

1. Release L1.

2. Press it again and drive **in the opposite direction** to the one that got you stuck.

3. If that does not free it, hold **L1 + L2 + R2** for a second to home the arm. Homing moves joint by joint, so it works from poses that ordinary Cartesian driving cannot escape.

⚠ Homing takes the **direct joint-space route with no collision checking**. From a twisted wrist that is harmless, but if the arm is somewhere awkward, watch the first second with a hand near the e-stop, and move the phantom clear first.

Symptom table

SYMPTOM	MOST LIKELY CAUSE	FIX
Arm does not move when L1 is held	You just homed it — teleop is deliberately disarmed	Release L1 and press it again. By design
Still does not move; LED green	Joint limit or singularity reached	Drive the other way, or home it — see above
Arm stops part-way through a move	Same. Not a fault	Drive the other way
Every command rejected, <code>Control Mode Is Not Tcp</code>	DobotStudio is connected and holding the arm	Close DobotStudio completely, then restart. The software claims control automatically at startup
Launch hangs with no output	Wrong IP, robot off, or no network	<code>ping -c3 192.168.5.1</code> . If it fails, <code>nmcli con up dobot-e6</code>
Constant rumble, arm clearly fine and not moving	The arm is silently refusing commands, so the software's idea of its pose drifts	Look for <code>ServoJ rejected</code> in the terminal. Full restart
Rumbles at rest right after startup	The arm was moved during the 25 s calibration window	Restart and keep hands off for the first 40 s
Red light	Alarm or collision detected	Full restart. If it returns immediately, power-cycle the arm
Controller does nothing at all	USB not detected	Unplug and replug. Check <code>ls /dev/input/js0</code> . Restart the program
Arm drifts with nobody touching anything	Stick drift on a worn controller	Release L1. Restart; if it persists the controller needs replacing
Collision alarms while scanning	Payload still declared as 0 kg	See §6

Does it need a restart?

SITUATION	RESTART?
Singularity or joint limit reached	No — drive back out
Rumbling constantly but the arm is fine	No — home it
Arm dead right after homing	No — release L1 and press it again
Emergency stop pressed	Yes — release it first
Red light / alarm state	Yes
Arm ignores the controller entirely	Yes
Network dropped, ping fails	Yes — and re-run <code>nmcli con up dobot-e6</code>

The full restart



```
```bash
```

### 1. Ctrl+C in the terminal running the launch

### 2. release the emergency stop if it is latched (rotate it)

```
nmcli con up dobot-e6 ping -c3 192.168.5.1
```

```
cd "/home/yz22/Documents/code/Dobot arm" source setup_dobot.bash python3 src/dobot_e6_hw/scripts/go_home.py --speed 5 ros2 launch dobot_e6_hw real_hw.launch.py robot_ip:=192.168.5.1
```

### 5. wait for the slow-flashing green light, then 25 s more for calibration

```
```
```

If a red light persists after a full restart, power-cycle the arm: **hold** the power button 1.5 s to switch off, wait ten seconds, then **tap** once to switch on.

Reading the launch output

| LINE | MEANS |
|------------------------|--|
| RequestControl → 0, {} | TCP control claimed. Good |
| EnableRobot → 0, {} | Arm enabled. Good |
| Servo started | Ready to drive |
| Calibration complete | Rumble threshold tuned. Safe to touch the controller |
| ServoJ rejected: ... | The arm is refusing motion. Check mode and alarms |
| RequestControl refused | Something else holds the arm — close DobotStudio |
| Robot PAUSEd | Controller stuck in pause; the software sends Stop() to flush it |

Diagnostic commands are in [§4 Running and stopping](#).

Running on Windows

A second, ROS-free teleop lives in `windows/` for machines that cannot run the Linux stack.

Why a separate program

The Linux stack converts the joystick's Cartesian twist into joint angles using **Movelt Servo**, and Movelt has no supported Windows build. But the robot's own **ServoP** command accepts a Cartesian pose directly and runs the inverse kinematics *inside the controller* — so Movelt is not needed at all.

| | LINUX STACK | WINDOWS_TELEOP.PY |
|----------------------|---|--|
| Inverse kinematics | Movelt Servo, on the PC | the robot's controller |
| Singularity handling | slow at κ 60, stop at κ 100 | none — see below |
| Haptic rumble | yes | no (needs <code>evdev</code> , Linux-only) |
| Simulation | Gazebo | no |
| Install | ROS 2 Humble + Movelt + colcon | <code>pip install pygame numpy requests</code> |

What you give up

⚠ **Singularity protection.** Movelt Servo measured the condition number and slowed the arm before it got into trouble. `ServoP` has no equivalent. The controller refuses poses it cannot reach, but its behaviour *approaching* a singularity is **untested on this arm**.

Four mitigations are built in: low default rates; the commanded pose re-seeded from the real pose on every deadman release; a 25 mm cap on how far the commanded pose may lead the actual one before the script stops; and a homing combo that escapes via joint space.

Install

```
pip install pygame numpy requests
```

Copy three files into one folder:

```
windows_teleop.py from windows/ dobot_api.py from src/dobot_e6_hw/scripts/ home_pose.py from src/dobot_e6_hw/scripts/
```

Set the wired adapter to a static `192.168.5.x` address, mask `255.255.255.0`, **no gateway**, connected to the robot's **LAN1** port. Check with `ping 192.168.5.1`.

First run — do not skip

Two things must be **measured**, not assumed. Guessing either has already caused a runaway on this project.

1. Gamepad indices

```
python windows_teleop.py --map
```

Does not touch the robot. Press each control, note the numbers, edit the config block at the top of the file. Windows enumerates a DualSense through a different backend, so the Linux indices **will** differ.

2. Cartesian axis signs

```
python windows_teleop.py --probe
```

Moves the arm 12 mm along each axis in turn, pausing between each. Watch which way it actually goes from where you stand, then set the sign constants.

This is necessary because **the Dobot frame is not the ROS frame**. At one recorded pose, ROS read the flange at `(-90, 232, 423)` mm while the robot reported `(-300, -91, 367)` mm — different origin *and* orientation. The signs worked out for the Linux stack do not carry over.

Driving

```
python windows_teleop.py
```

Same layout as §5: deadman held throughout, three-button combo for homing, `Ctrl+C` to disable and exit.

⚠ Everything in §1 [Safety](#) and §6 [Contact force](#) applies unchanged. The arm is still position controlled and still does not yield.

Reference

Robot specification

| | |
|---------------------|---|
| Maximum load | 750 g including all tooling |
| Maximum reach | 450 mm radius |
| Repeatability | ± 0.1 mm |
| Maximum tool speed | 0.5 m/s (this setup uses 0.03 m/s) |
| Maximum joint speed | 120°/s |
| Mass | 7.2 kg |
| Power | 100–240 V AC, 50/60 Hz |
| Joint travel | J1 $\pm 360^\circ$, J2 $\pm 135^\circ$, J3 $\pm 154^\circ$, J4 $\pm 160^\circ$, J5 $\pm 173^\circ$, J6 $\pm 360^\circ$ |

Network

| | |
|-------------------|---|
| Robot LAN1 | <code>192.168.5.1</code> |
| Laptop wired port | <code>enp59s0</code> at <code>192.168.5.10</code> |
| Network profile | <code>dobot-e6</code> — does not auto-connect; bring it up by hand |
| Dashboard port | 29999 (commands) |
| Feedback port | 30004 (joint states, 8 ms) |

Home pose

Chosen so the flange sits **parallel to the bench** while keeping a wide singularity margin ($\kappa \approx 15.9$, versus the Servo stop threshold of 100):

```
J1 0.00° J4 -8.02° J2 -19.24° J5 -90.00° J3 117.25° J6 0.00°
```

Defined once in `src/dobot_e6_hw/scripts/home_pose.py` and shared by `go_home.py` and the controller homing key, so the two cannot disagree.

Where things live

| | |
|----------------------------|---|
| Workspace | /home/yz22/Documents/code/Dobot arm |
| Joystick mapping | src/dobot_e6_bringup/config/joy_params.yaml |
| Home pose + motion profile | src/dobot_e6_hw/scripts/home_pose.py |
| Standalone homing | src/dobot_e6_hw/scripts/go_home.py |
| Robot interface | src/dobot_e6_hw/scripts/dobot_tcp_node.py |
| Teleop mapping logic | src/dobot_e6_bringup/scripts/joy_to_servo.py |
| Haptics | src/dobot_e6_bringup/scripts/haptic_feedback.py |
| Windows teleop | windows/windows_teleop.py |

Tuning for the stand

All in `joy_params.yaml`. Edit, then restart the program — **no rebuild needed**, the workspace is symlink-installed.

| SETTING | NOW | EFFECT |
|--------------------------------|------|--|
| <code>scale_linear</code> | 0.03 | Translation speed, m/s |
| <code>scale_angular</code> | 0.2 | Roll and pitch rate, rad/s |
| <code>scale_angular_yaw</code> | 0.3 | Yaw rate, rad/s |
| <code>scale_turbo</code> | 1.8 | Turbo multiplier, operator only . Set to 1.0 to remove turbo entirely |

Servo thresholds

| SETTING | VALUE | MEANING |
|--|---------|---|
| <code>lower_singularity_threshold</code> | 60 | Start decelerating |
| <code>hard_stop_singularity_threshold</code> | 100 | Stop |
| <code>joint_limit_margin</code> | 0.1 rad | Stop 5.7° before a joint limit |
| <code>check_collisions</code> | false | Movelt collision checking is off |
| Rumble threshold | ≈25 | Auto-calibrated at every launch |

Rebuilding

Only needed if you add or rename a file, or change `CMakeLists.txt`:

```
bash cd "/home/yz22/Documents/code/Dobot arm" source /opt/ros/humble/setup.bash source .venv/bin/activate colcon build --symlink-install
```

Editing Python or YAML needs **no** rebuild — just restart the program.